An adapter is useful if your code has any or all of the following qualities:

- The external service will be accessed from multiple points in your code.
- The interaction with the external service has logic of its own, such as authentication or type changing or common sets of options.
- There's a mismatch between the language or metaphor of the API and the domain terms and structures of your code.

The Twitter example doesn't have the first quality, at least not yet. You do have the second feature: the adapter needs to manage a Twitter client object that nothing else in the application needs to care about or be aware of. It also manages an argument to the Twitter API: the :bigger argument, which specifies the size of the image you want to download.

Whether there is a mismatch between the API and the code is a matter of interpretation, but I think there is. At the very least, the Twitter client exposes a lot of data that the application doesn't care about, so limiting access to the full set of Twitter data seems like a good idea.

My experience with adapters like the one we've written is that they tend to attract functionality as you use them, with the side effect that it's much easier for a full range of complexity to be available at each use. For example, if you allow the adapter to take an argument to image_url to represent the size, then that ability is automatically available whenever the adapter is used. Attracting functionality is especially valuable for security and error handling, which are easy to leave off when you're creating each connection separately.

Adapter Tests

The adapter tests work between the adapter and the server, using VCR as a medium:

external/02/spec/models/avatar_adapter_spec.rb

```
require "rails_helper"

describe AvatarAdapter do

  it "accurately receives image url", :vcr do
    user = instance_double(User, twitter_handle: "noelrap")
    adapter = AvatarAdapter.new(user)
    url = "http://pbs.twimg.com/profile_images/40008602/head_shot_bigger.jpg"
    expect(adapter.image_url).to eq(url)
  end

end
```